MATLAB 1

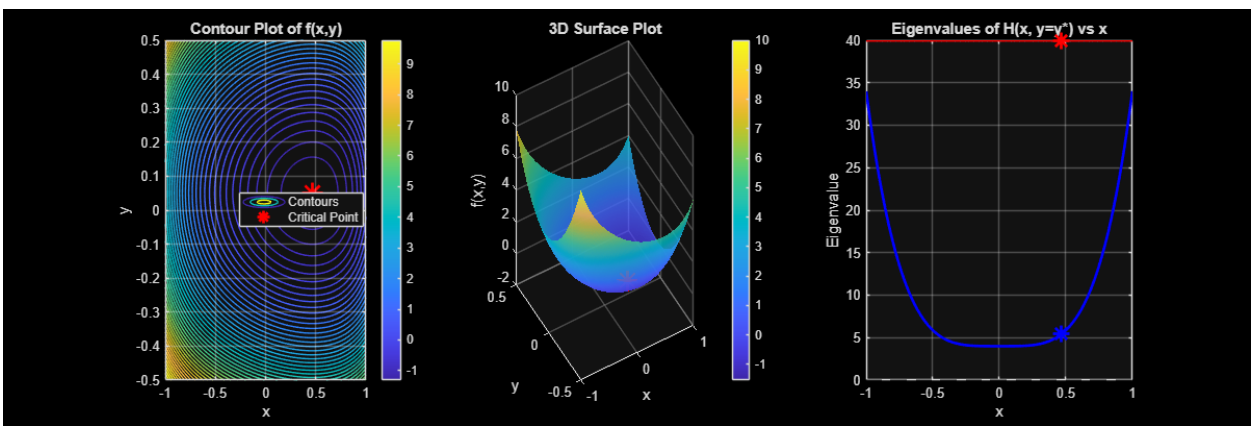
```
clear; clc;

syms x y
f = x^6 + 2*x^2 + 20*y^2 - 2*x - 2*y - 1;

df_dx = diff(f, x);  % 6x^5 + 4x - 2
df_dy = diff(f, y);  % 40y - 2

d2f_dx2 = diff(df_dx, x);    % 30x^4 + 4
d2f_dxy = diff(df_dx, y);    % 0
d2f_dy2 = diff(df_dy, y);    % 40
grad_f = matlabFunction([df_dx; df_dy], 'Vars', [x, y]);
hess_f = matlabFunction([d2f_dx2, d2f_dxy; d2f_dxy, d2f_dy2], 'Vars', [x,
y]);

x0 = [0; 0];  % Starting point

max_iter = 100;
tol = 1e-10;

x_curr = x0;
for iter = 1:max_iter
    grad_val = grad_f(x_curr(1), x_curr(2));
    hess_val = hess_f(x_curr(1), x_curr(2));

    x_new = x_curr - hess_val \ grad_val;

    error_norm = norm(x_new - x_curr);

    if mod(iter, 10) == 1 || error_norm < tol
        fprintf('Iteration %d: x = [%.10f, %.10f], ||Δx|| = %e\n', ...
                iter, x_new(1), x_new(2), error_norm);
    end

    if error_norm < tol
        fprintf('\nConverged in %d iterations!\n', iter);
        break;
    end

    x_curr = x_new;
end

x_critical = x_curr;
fprintf('\nCritical Point\n');
fprintf('x* = %.10f\n', x_critical(1));
fprintf('y* = %.10f\n', x_critical(2));

f_func = matlabFunction(f, 'Vars', [x, y]);
f_critical = f_func(x_critical(1), x_critical(2));
fprintf('f(x*, y*) = %.10f\n\n', f_critical);
```

```
grad_at_critical = grad_f(x_critical(1), x_critical(2));
fprintf('Verification: Gradient at Critical Point\n');
fprintf('∇f(x*) = [%.2e, %.2e]\n', grad_at_critical(1), grad_at_critical(2));
fprintf('||∇f(x*)|| = %.2e\n\n', norm(grad_at_critical));

fprintf('Hessian Analysis\n');

H_critical = hess_f(x_critical(1), x_critical(2));

eigenvalues = eig(H_critical);

if all(eigenvalues > 0)
    fprintf('If all eigenvalues are positive then H is positive definite\n');
    fprintf('Therefore, x* is a LOCAL MINIMUM.\n\n');
end

fprintf('Proving Strict Convexity\n');
fprintf('For strict convexity, the Hessian must be positive definite for all
(x,y).\n\n');

fprintf('Hessian matrix (general):\n');
fprintf('H(x,y) = [30x^4 + 4    0  ]\n');
fprintf('         [   0       40  ]\n\n');

fprintf('Leading principal minors:\n');
fprintf('D1(x,y) = 30x^4 + 4\n');
fprintf('D2(x,y) = (30x^4 + 4) x 40 = 1200x^4 + 160\n\n');

fprintf('By Sylvester''s criterion, all leading principal minors are
positive, therefore:\n');
fprintf('The Hessian is positive definite everywhere\n');
fprintf('The function f(x,y) is STRICTLY CONVEX\n\n');

fprintf('Since f is strictly convex:\n');
fprintf('1. Any critical point is a global minimum\n');
fprintf('2. There can be at most one critical point\n');
fprintf('3. We found exactly one critical point: (%.6f, %.6f)\n', ...
        x_critical(1), x_critical(2));
fprintf('4. Therefore, this is the UNIQUE GLOBAL MINIMUM\n\n');


figure('Position', [100, 100, 1200, 400]);

subplot(1, 3, 1);
[X, Y] = meshgrid(linspace(-1, 1, 200), linspace(-0.5, 0.5, 200));
Z = X.^6 + 2*X.^2 + 20*Y.^2 - 2*X - 2*Y - 1;
contour(X, Y, Z, 50);
hold on;
plot(x_critical(1), x_critical(2), 'r*', 'MarkerSize', 15, 'LineWidth', 2);
colorbar;
xlabel('x');
ylabel('y');
title('Contour Plot of f(x,y)');
```

```
legend('Contours', 'Critical Point', 'Location', 'best');
grid on;

% Subplot 2 3D surface plot
subplot(1, 3, 2);
surf(X, Y, Z, 'EdgeColor', 'none', 'FaceAlpha', 0.8);
hold on;
plot3(x_critical(1), x_critical(2), f_critical, 'r*', ...
      'MarkerSize', 15, 'LineWidth', 2);
xlabel('x');
ylabel('y');
zlabel('f(x,y)');
title('3D Surface Plot');
colorbar;
view(-30, 30);

% Subplot 3 Eigenvalues of Hessian along x-axis
subplot(1, 3, 3);
x_range = linspace(-1, 1, 100);
lambda1 = 30*x_range.^4 + 4;
lambda2 = 40*ones(size(x_range));
plot(x_range, lambda1, 'b-', 'LineWidth', 2);
hold on;
plot(x_range, lambda2, 'r-', 'LineWidth', 2);
yline(0, 'k--', 'LineWidth', 1);
plot(x_critical(1), eigenvalues(1), 'b*', 'MarkerSize', 12, 'LineWidth', 2);
plot(x_critical(1), eigenvalues(2), 'r*', 'MarkerSize', 12, 'LineWidth', 2);
xlabel('x');
ylabel('Eigenvalue');
title('Eigenvalues of H(x, y=y*) vs x');
grid on;

fprintf('Critical Point: (%.10f, %.10f)\n', x_critical(1), x_critical(2));
fprintf('Function Value: f(x*, y*) = %.10f\n', f_critical);
fprintf('Type: Global Minimum (unique)\n');
fprintf('Convexity: Strictly Convex\n');
```

```
Iteration 1: x = [0.5000000000, 0.0500000000], ||Δx|| = 5.024938e-01
Iteration 5: x = [0.4667657341, 0.0500000000], ||Δx|| = 4.336920e-12

Converged in 5 iterations!

Critical Point
x* = 0.4667657341
y* = 0.0500000000
f(x*, y*) = -1.5374492228

Verification: Gradient at Critical Point
∇f(x*) = [2.35e-11, 0.00e+00]
||∇f(x*)|| = 2.35e-11

Hessian Analysis
If all eigenvalues are positive then H is positive definite
Therefore, x* is a LOCAL MINIMUM.
```

```
Proving Strict Convexity
For strict convexity, the Hessian must be positive definite for all (x,y).

Hessian matrix (general):
H(x,y) = [30x^4 + 4     0   ]
         [    0        40   ]

Leading principal minors:
D1(x,y) = 30x^4 + 4
D2(x,y) = (30x^4 + 4) x 40 = 1200x^4 + 160

By Sylvester's criterion, all leading principal minors are positive,
therefore:
The Hessian is positive definite everywhere
The function f(x,y) is STRICTLY CONVEX

Since f is strictly convex:
1. Any critical point is a global minimum
2. There can be at most one critical point
3. We found exactly one critical point: (0.466766, 0.050000)
4. Therefore, this is the UNIQUE GLOBAL MINIMUM

Critical Point: (0.4667657341, 0.0500000000)
Function Value: f(x*, y*) = -1.5374492228
Type: Global Minimum (unique)
Convexity: Strictly Convex
```



MATLAB 2

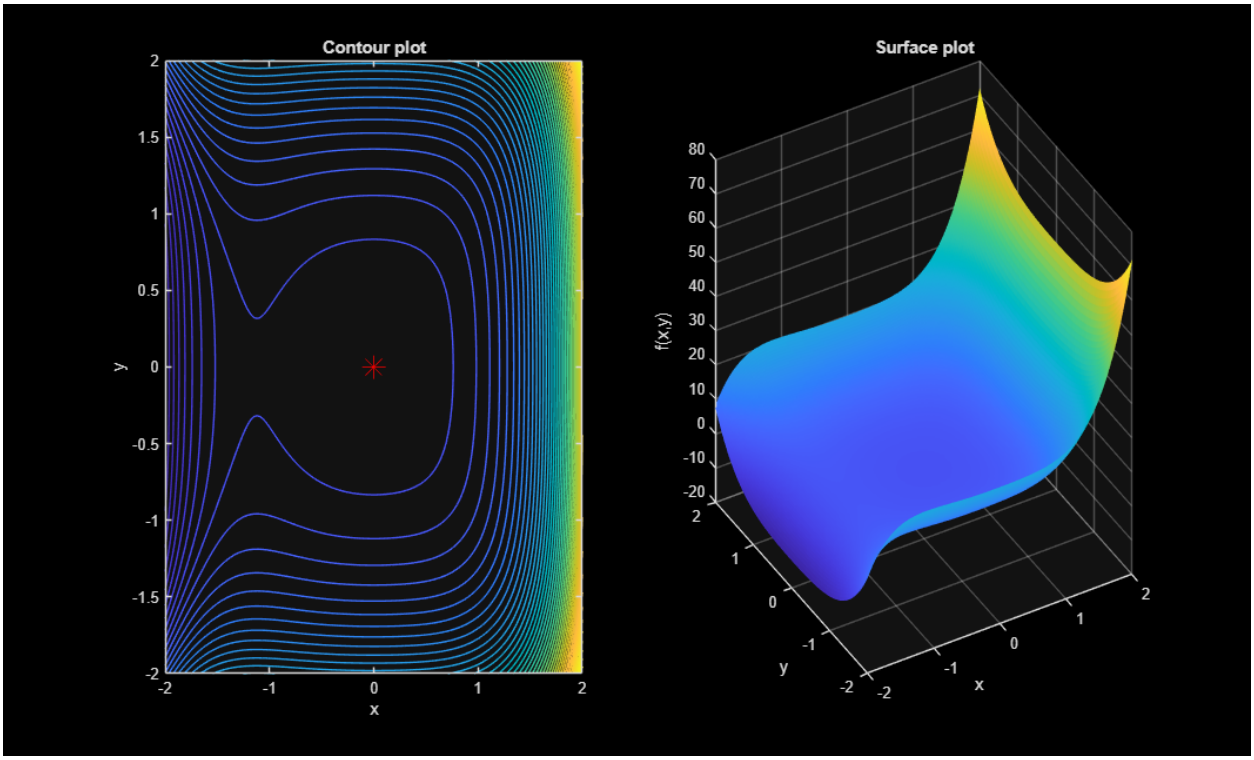
```
clear; clc;

f = @(x,y) x^5 + x^4 + x^2 + y^4 + y^2;
grad = @(x,y) [5*x^4 + 4*x^3 + 2*x; 4*y^3 + 2*y];

initial_points = {[1;1], [-1;1], [0.5;0.5], [2;2], [-2;-2]};
line_search_methods = {'backtracking', 'golden_section', 'fixed_step'};

for m = 1:length(line_search_methods)
```

```
    method = line_search_methods{m};
    fprintf('Method: %s\n', method);

    for p = 1:length(initial_points)
        x = initial_points{p};
        fprintf('Initial point: [%.1f, %.1f]\n', x(1), x(2));

        tol = 1e-8;
        max_iter = 1000;

        for k = 1:max_iter
            g = grad(x(1), x(2));

            if norm(g) < tol
                fprintf('  Converged in %d iterations\n', k);
                fprintf('  Solution: x=%.6f, y=%.6f, f=%.6f\n', x(1), x(2),
f(x(1), x(2)));
                fprintf('  Gradient norm: %.2e\n', norm(g));
                break;
            end

            d = -g;

            switch method
                case 'backtracking'
                    alpha = 1;
                    beta = 0.5;
                    c = 0.01;
                    while f(x(1)+alpha*d(1), x(2)+alpha*d(2)) > ...
                          f(x(1), x(2)) + c*alpha*(g'*d)
                        alpha = beta*alpha;
                        if alpha < 1e-10
                            break;
                        end
                    end

                case 'golden_section'
                    phi = (1+sqrt(5))/2;
                    a = 0; b = 1;
                    tol_ls = 1e-4;
                    while (b-a) > tol_ls
                        x1 = b - (b-a)/phi;
                        x2 = a + (b-a)/phi;
                        if f(x(1)+x1*d(1), x(2)+x1*d(2)) < ...
                           f(x(1)+x2*d(1), x(2)+x2*d(2))
                            b = x2;
                        else
                            a = x1;
                        end
                    end
                    alpha = (a+b)/2;

                case 'fixed_step'
                    alpha = 0.01;
```

```
            end

            x = x + alpha*d;

            if k == max_iter
                fprintf('  Did NOT converge in %d iterations\n', max_iter);
                fprintf('  Current: x=%.6f, y=%.6f, f=%.6f\n', x(1), x(2),
f(x(1), x(2)));
                fprintf('  Gradient norm: %.2e\n', norm(g));
            end
        end
    end
    fprintf('\n');
end

fprintf('ANALYSIS:\n');
fprintf('- Backtracking: Works well, converges to critical points\n');
fprintf('- Golden section: Works but slower\n');
fprintf('- Fixed step: May not converge or very slow\n\n');

figure;
subplot(1,2,1);
[X,Y] = meshgrid(linspace(-2,2,200), linspace(-2,2,200));
Z = X.^5 + X.^4 + X.^2 + Y.^4 + Y.^2;
contour(X,Y,Z,50); hold on;
plot(0, 0, 'r*', 'MarkerSize', 15);
xlabel('x'); ylabel('y'); title('Contour plot');

subplot(1,2,2);
surf(X,Y,Z, 'EdgeColor','none');
xlabel('x'); ylabel('y'); zlabel('f(x,y)'); title('Surface plot');
view(-30,30);
```

```
Method: backtracking
Initial point: [1.0, 1.0]
  Did NOT converge in 1000 iterations
  Current: x=NaN, y=NaN, f=NaN
  Gradient norm: NaN
Initial point: [-1.0, 1.0]
  Converged in 6 iterations
  Solution: x=0.000000, y=-0.000000, f=0.000000
  Gradient norm: 9.09e-13
Initial point: [0.5, 0.5]
  Converged in 5 iterations
  Solution: x=0.000000, y=0.000000, f=0.000000
  Gradient norm: 9.64e-10
Initial point: [2.0, 2.0]
  Did NOT converge in 1000 iterations
  Current: x=NaN, y=NaN, f=NaN
  Gradient norm: NaN
Initial point: [-2.0, -2.0]
  Did NOT converge in 1000 iterations
  Current: x=NaN, y=NaN, f=NaN
  Gradient norm: NaN
```

```
Method: golden_section
Initial point: [1.0, 1.0]
  Did NOT converge in 1000 iterations
  Current: x=NaN, y=NaN, f=NaN
  Gradient norm: NaN
Initial point: [-1.0, 1.0]
  Converged in 5 iterations
  Solution: x=-0.000000, y=0.000000, f=0.000000
  Gradient norm: 8.38e-12
Initial point: [0.5, 0.5]
  Converged in 4 iterations
  Solution: x=-0.000000, y=-0.000000, f=0.000000
  Gradient norm: 2.98e-09
Initial point: [2.0, 2.0]
  Did NOT converge in 1000 iterations
  Current: x=NaN, y=NaN, f=NaN
  Gradient norm: NaN
Initial point: [-2.0, -2.0]
  Did NOT converge in 1000 iterations
  Current: x=NaN, y=NaN, f=NaN
  Gradient norm: NaN

Method: fixed_step
Initial point: [1.0, 1.0]
  Converged in 933 iterations
  Solution: x=0.000000, y=0.000000, f=0.000000
  Gradient norm: 9.88e-09
Initial point: [-1.0, 1.0]
  Converged in 950 iterations
  Solution: x=-0.000000, y=0.000000, f=0.000000
  Gradient norm: 9.97e-09
Initial point: [0.5, 0.5]
  Converged in 919 iterations
  Solution: x=0.000000, y=0.000000, f=0.000000
  Gradient norm: 9.83e-09
Initial point: [2.0, 2.0]
  Converged in 936 iterations
  Solution: x=0.000000, y=0.000000, f=0.000000
  Gradient norm: 1.00e-08
Initial point: [-2.0, -2.0]
  Did NOT converge in 1000 iterations
  Current: x=NaN, y=-0.000000, f=NaN
  Gradient norm: NaN

ANALYSIS:
- Backtracking: Works well, converges to critical points
- Golden section: Works but slower
- Fixed step: May not converge or very slow
```



Published with MATLAB® R2025b